

write-critical-guide

A User's Manual

Working material for someone who has to judge a paper.

Everything a critical reader needs, and no verdict.

Last revised: September 11, 2026

torsor lab

torsor.org

Contents

Preface: Someone Has to Judge It	iii
1 Findings, Never a Verdict	1
The sibling genre	1
Three rules	2
Grades, and what they measure	4
What it is not	4
2 How It Reads a Paper	5
Starting it	5
Which text is under examination	6
Three source situations	6
The reading pass	7
Four sweeps, in parallel	8
Refutation, then promotion	10
When repair has more than one route	11
3 The Ledger and the Package	13
Categories	13
Four attributes, kept independent	15
Anchors	15
The generator	16

Writing notes	17
What is written by hand	18
The finished package	19
The gates	20
4 Long Work, and Corrections	21
When one pass isn't enough	21
Correcting a finished guide	24
A Quick Reference	27
Invocation	27
The generator	27
Package layout	28
Ledger fields	29
Where fields appear	30
Source cases	31
LaTeX hazards, already handled	31
Building the guide PDF	32
Chunked examinations	33
Words the house does not use	33

Preface: Someone Has to Judge It

Let's be honest with each other. You said yes to refereeing a paper, and the paper is fifty pages long, and three of those pages are a computation you would have to redo yourself to believe.

Or a committee has asked what you think of work outside your area. Or a student wants to build on a result and you'd like to know, before they spend a year on it, whether the result is actually there.

The labor is the same in all three cases and it is mostly invisible. Read the paper closely enough to find what's wrong with it. Check that the cited lemma says what the paper uses it for. Work out whether the gap in Section 4 is a gap or just a compression. Establish what was already known, so you can tell what is new here. And then — the part that costs the most and shows the least — convince yourself that each thing you are about to call an error really is one, because you are going to put your name on it.

`write-critical-guide` does that labor and hands you the result. It is working material for a person who has to assess a paper: the paper read closely, its context established, everything questionable found and located and verified, with repairs where repairs exist.

What it does not do is decide. The words *accept*, *reject*, *major revision* appear nowhere in what it produces, as judgments about the paper. That restraint is the point rather than a limitation. The decision carries your name, and material that arrives at a verdict invites you to ratify a judgment you did not make. Findings, ordered by consequence, and nothing that presumes the decision.

Two properties are worth knowing before you run it. It records what checked out, not only what failed — if it verified thirty citations and nine were wrong, it says thirty, and it says what it did not check, because that is exactly where your own work begins. And it tries to refute every serious finding before writing it down, by sending it to independent skeptics whose job is to show the paper is right. A defect reported that isn't there costs the authors weeks and costs you your credibility; a missed comma costs a reader three seconds. The errors are not symmetric and the process doesn't treat them as though they were.

It runs unattended. Point it at a paper and it goes to completion without asking you anything.

Four chapters follow. Chapter 1 is what the genre is and the three rules that govern it. Chapter 2 is how it reads a paper — the sweeps, run in parallel, and the refutation pass they feed. Chapter 3 is the ledger every artifact is generated from, which is the piece of engineering that keeps the prose and the annotations from drifting apart. Chapter 4 covers long works reviewed in chunks, and how to correct a guide after it's finished. The appendix has the command lines and the schema.

Findings, Never a Verdict

A critical guide is the material a judgment gets written from, not the judgment.

Hold that distinction and everything else in this manual follows from it. Lose it and the output becomes a referee report with no name on it, which is the one thing it must not be.

The sibling genre

The quickest way to see what belongs in a critical guide is to put it beside the reading guide that `write-paper-guide` produces. Same family, same close reading underneath, same voice — opposite job.

A reading guide is expository. It takes the paper as given and helps someone through it. Where the paper compresses, it expands, so the reader can keep moving.

A critical guide is evaluative. It asks whether the paper holds. Where the paper compresses, it stops and says so: what is missing, how much it matters, and whether the gap can be closed.

So the test for any sentence is which document it belongs in. If it explains the paper to someone who wants to understand it, that's the reading guide. If it weighs the paper for someone who has to judge it, it belongs here.

The two are not alternatives. A reading guide is the *input* to a critical guide — the first thing the skill does is commission one, at proof depth, for a reader who is refereeing. The critical work is done on top of that reading, not instead of it.

Three rules

Findings, never a disposition

Accept, accept with revision, major revision, minor revision, reject: none of these appears as a judgment about the paper, anywhere in the package. Nor does anything that writes your report for you — no *I recommend*, no *this reviewer finds*.

What appears instead: what is wrong, how consequential it is, and what would repair it. Ordering the issues by consequence is as close to a verdict as it comes, and that ordering is the most useful thing the summary does.

This isn't modesty about the mathematics. It follows from what the document is. The disposition is yours to reach and your name goes on it.

Try to refute a finding before writing it down

Every finding that would be graded *major* goes to three independent skeptics before it enters the ledger. Each gets the paper and the finding and is told to **refute** it — to find the argument the finding claims is missing, the hypothesis it claims is undischarged, the reading under which the notation is consistent. They are told to default to *refuted* when genuinely uncertain, and none of them is told what the others concluded.

Two or more refute it and the finding is dropped, or demoted to minor if a weaker true version survives. The drop is recorded with

its refutation rather than silently discarded, so you can see what was considered and rejected.

Claims about the literature get the same treatment whatever their grade, because nothing else in the pipeline checks them and they are the most quotable sentences in the package. “The first result in arbitrary dimension”; “X proved only the surface case”; “the technique originates with Y.” Being confidently wrong about priority in front of the author whose priority it is costs you more than a missed lemma would.

Every priority or duplication charge gets a chronology check, without exception: when each work first appeared *publicly* — preprint date, not publication date — whether the earlier work is where the result actually originates or is itself citing someone earlier, and whether the charge is even coherent in time. A duplication charge that dissolves on dates alone is the commonest false positive in the whole pipeline.

The effort spent checking scales to what being wrong would cost, not to how likely the error seems. A finding whose consequence is that an author adds a sentence gets one pass. A finding whose consequence is that you accuse someone of taking another person’s result without credit gets as many as it takes.

Record what checked out, not only what failed

A list of nine problems reads as the complete result of an examination. It usually isn’t.

So the package says what was verified and came out clean — a table of citations checked, what each is used for, which held. And it says explicitly what was *not* checked, with reasons. If the long computation in Section 6 wasn’t re-derived, that’s stated.

You cannot tell a clean bill of health from an unopened box unless someone tells you which it was. Where the coverage ends is where your own work starts.

Grades, and what they measure

Three of them, and they grade consequence rather than length.

Grade	Meaning
major	affects correctness, or a reader cannot follow without reconstructing the argument
minor	should be fixed, but the reader gets there
trivial	copy-editing

A misprinted subscript that makes two displays contradict each other is major. A paragraph of clumsy prose is trivial. The item that takes three words to state can be the one that breaks the theorem.

Alongside the grade sits a qualifier where a judgment call needs recording — *trivial (but should be reconciled)* is a different instruction to an author than *trivial*.

What it is not

Not a referee report. It carries no signature and reaches no disposition.

Not a reading guide — it assesses and itemizes rather than orienting a newcomer.

Not a rewrite. Where the authors’ prose is defensible and merely not yours, it is left alone, and a preference is never reported as an error.

Not a survey. Context appears in service of assessing this paper.

How It Reads a Paper

Point it at a paper and walk away. This chapter is what happens while you're gone.

Starting it

```
/write-critical-guide ~/refereeing/descent
.pdf
/write-critical-guide arXiv:0000.00000
/write-critical-guide ~/refereeing/descent
/
```

Given a folder, it looks for a single PDF or LaTeX source and uses that. Given nothing, it looks in the working directory. If there are several candidates and nothing distinguishes them, it says which it found and stops — that is the only thing that halts a run. Everything else has a default, and it takes the default and keeps going.

Unattended is the design, not a convenience. The run dispatches a dozen or more agents across the sweeps and the refutation passes, and a pipeline that paused for confirmation between them would take a day of your attention to produce what it otherwise produces while you do something else.

Which text is under examination

First it settles what the findings are *about*, and this matters more than it sounds.

Where a PDF exists, the PDF is the artifact. Source — the authors' LaTeX, an arXiv tarball, a repository — is an aid: for exact quotation, for line numbers, for placing annotations, for corroborating what the compiled document shows.

The test is whether a thing appears in the compiled paper. An author's marker that a macro renders, a cross-reference set as ??, a sentence that stops mid-clause, a statement with no proof: all of these print, so all are in scope. Commented-out source does not print, and is out of scope entirely — not reported, not reasoned from, not referred to.

That last rule is stricter than it first appears, and it's right. A passage may be commented out because it was wrong, because it was superseded, or because it was never meant to survive, and none of that is knowable from outside. Findings drawn from working notes are findings about a person's process rather than their work, and they're unsound in a way that is hard to see afterwards. A statement whose proof is commented out is a statement with no proof — that is the finding, and it is a finding about the printed paper.

Three source situations

Where the *annotations* get inserted is a separate question, and there are three cases. Which one you're in is recorded in the package README.

Case	Situation
A	The authors' source is available and is the text under examination. Annotated directly; locations cite that source by line, plus the paper's own numbering.
B	The submission is a PDF and a public source exists for the same work — the common journal case. The public source is annotated, and every annotated passage is checked against the submitted PDF first. Locations in the prose refer to the submission by page and line.
C	A PDF and nothing else. There are no annotated sources; the package is the prose documents and the typeset guide, and the README says so rather than leaving it to be inferred.

Case B runs two location systems at once, and mixing them produces notes pointing at passages that aren't there. A note about a sentence the authors already fixed in the public version is worse than no note. Watch for a cover sheet, too — Editorial Manager and its relatives prepend one, so the printed page number and the PDF page number differ by one.

The reading pass

You cannot assess a paper you have not read, and the reading is where the findings come from. So the first real phase commissions a reading guide: *write-paper-guide*, run unattended, at proof depth, calibrated to a reader who is refereeing — an active researcher in an adjacent field, comfortable with the machinery, without the local folklore.

Alongside that guide it keeps a running ledger of friction. Every time the reading has to supply a step the paper leaves out, reconstruct an argument to follow it, guess which of two things a symbol means, or

look up whether a cited result says what it's used for, an entry goes in: where it happened, quoted verbatim; what the paper says; what had to be supplied; and whether the gap looks fillable.

That ledger is the reason for the reading. A guide written to proof depth has to touch every place the paper is thin, and writing down the friction as it happens is cheaper and more honest than trying to recall it afterwards.

If a guide already exists for this paper, it's read and used rather than rebuilt.

Four sweeps, in parallel

Then independent agents sweep the paper, one per category, each writing its own file. **None of them sees another's output.** Uncorrelated readings are the point; a shared draft would collapse them into one.

Sweep	What it does
Typographical	Reads start to finish for grammar, usage, mathematical typesetting, and inconsistency held against the paper's own practice elsewhere. Does not report a house preference as an error.
Mathematical	Seeds from the reading ledger, then reads independently for what it missed. For each item: what the paper claims, what its argument establishes, what is missing between them, and whether the gap can be filled — and if it can, fills it.
References	Enumerates every citation and checks four things per citation: does the cited statement say what it's used for; are its hypotheses satisfied at the point of use; is the locator right; is the record itself correct.
Context	Establishes where the paper sits, and states the trade against the nearest prior work precisely — where it's stronger, where weaker, where the two simply aren't comparable.

Those four suit most pure-mathematics papers. They're a starting set, not a schema: a paper with substantial numerics or released code earns a computational sweep, a long computation may be worth re-deriving independently, and two sweeps that would return nearly the same thing get merged. The sweeps that run determine the categories in the ledger, not the other way round.

Every reported item must carry a verbatim quotation from the source, long enough to locate the passage uniquely. That quotation becomes the annotation anchor, and an item without one cannot be placed. This is why retyping from a rendered PDF is a mistake — the quotation has to match the file.

What the mathematical sweep produces besides findings

A dependency map: which results rest on which, and in particular which findings are independent defects and which are consequences of an earlier one.

A theorem whose proof fails only because a lemma it cites fails is one error, not two. Presenting it as two makes the paper look worse than it is, and a reader working from that list will grade the paper on a count that isn't real.

What the reference sweep reports

Three things, all of them. The citations with problems. A table of the citations that came out clean, saying what each is used for. And an explicit list of what could not be verified, with the reason.

Anything central cited to a work that is unpublished, "in preparation," or otherwise unavailable to a reader gets flagged, because that's a question you will have to take a position on whether or not you want to.

Refutation, then promotion

Every candidate major finding then goes through the adversarial pass described in Chapter 1: three skeptics, told to refute it, defaulting to refuted when uncertain.

What survives enters the ledger carrying its **confidence basis**, and the distinction is kept because the two claims are not the same:

Basis	What it claims
direct-check	carried out in full; the expectation, and unmarked per item
assessment reported	the route looks viable but has not been written out taken from the source, not independently re-derived

Then the promotion check runs the other way: any finding graded minor that the dependency map shows a main theorem resting on is re-examined, and promoted if the dependency is real.

When repair has more than one route

Sometimes a confirmed major issue admits several plausible repairs — add a hypothesis here, or restrict the generality there, and the theorem survives either way in different amended forms. When that happens a separate dossier gets written: one agent per route, each saying what the hypothesis is, what it costs in generality, exactly which of the paper’s results it restores and in what form, and whether it is *sharp* or merely sufficient. A synthesis follows, recommending an order of preference.

That recommendation is about mathematics, not about publication, so it doesn’t touch the first rule.

When there’s one obvious fix, none of this fires — the fix goes in the issue’s own repair field and the dossier is skipped. Branches are never manufactured to fill it.

Branching never nests. A branch is a genuine choice among repair routes, and choices that compose cost their product rather than their sum. If a second choice arises inside a branch whose options depend on which branch you’re in, it isn’t allowed to branch: correctness settles it first, and consistency with the sur-

rounding branch is only a tiebreaker between options already known to be sound.

And if the branch point is foundational — if resolving it changes what a large part of the paper says — the repair is not determined at all. That gets reported as a critical issue rather than patched around, because a document that silently embodies one arbitrary resolution misleads.

Many independent branch points is itself a finding: it says the repair is underdetermined, and the guide says so plainly instead of presenting a menu.

The Ledger and the Package

Everything converges on one file. `issues.yaml` holds every finding, and both the point-by-point issue list and the annotated copies of the paper are *generated* from it.

That direction is the whole design. Maintaining the prose and the annotation anchors side by side is what lets them drift, and a note pointing at the wrong passage is worse than no note at all. Here an anchor that no longer matches its source is a hard error rather than a silent mismatch.

Categories

Nothing in the tooling knows what a category is. They come from the ledger, and the tags, colours, output files, and annotated copies all follow from what you declare.

The default three fit most pure-mathematics papers, and they work because they split by what the author has to *do* rather than by subject matter:

Category	Tag	What it asks of the author
Mathematical	M	think — supply an argument, discharge a hypothesis, rename a symbol
References	R	look something up — a locator, a record, a missing citation
Typographical	T	retype — grammar, usage, typesetting

An author works through those in different sittings and often in different moods, which is why the split earns its keep.

Depart from them when the paper warrants it. Substantial numerics or released code may deserve a computational category; a paper whose real problems are organizational may want exposition kept separate from typography; a survey may need attribution. Two constraints, and they're the only ones. A category that would hold one or two items is not a category — fold it into a neighbour, since every category costs the reader another PDF to open. And categories partition: every issue belongs to exactly one, filed where the *action* lies, with a cross-reference from the other.

Colours are any `xcolor` name, used as a light fill and a darker border. Keep the hues far apart so a reader flipping between PDFs can tell them apart at a glance, and keep each category's colour stable across revisions. After red and green and blue, reach for orange, violet, teal, brown. Yellow and lime are invisible at that tint.

Four attributes, kept independent

Attribute	What it carries
Tag	category and identity: [M-7], [R-3]. The same tag appears in the issue list, the summary's cross-references, and the inline note, so nothing needs translating. Stable once assigned — a dropped issue retires its number rather than renumbering the rest.
Grade	what it costs: major, minor, trivial. By consequence, not by length.
Dependency	<code>depends_on</code> : [M-2] — this item is not an independent defect; its status follows from how another one resolves.
Confidence	how it is known: a direct check, a feasibility assessment, or something reported and not re-derived.

Collapsing any two of these loses information the author needs. In particular, a dependent item is still listed — the author needs to know which downstream statements are affected — but it is never presented as a separate problem.

Anchors

One rule decides whether the package builds at all:

An anchor must be a verbatim substring of exactly one line of the file being annotated.

Copy it out of the `.tex`, macros and all. Retyping from the rendered PDF produces an anchor that never matches, because what you see set on the page is not what's in the file.

Not found: lengthen or correct it against the file. Matching several

lines: lengthen it until unique, or ask for a particular occurrence when the repetition is real. An anchor on a line that ends in the middle of inline math is fine — the insertion point advances past any straddling math or display, which is what keeps a note from landing inside an equation and producing a `Missing $ inserted` error pointing at a line nobody wrote.

Check anchors before any prose gets written. Fixing an anchor is cheap; fixing it after four documents quote the passage is not.

```
python3 annotate_tex.py check -v
```

The generator

`annotate_tex.py` builds the generated half of the package from the ledger.

```
# anchors only; writes nothing
python3 annotate_tex.py check -v

# one annotated .tex per category
python3 annotate_tex.py annotate -o .

# the point-by-point Markdown issue list
python3 annotate_tex.py issues-md -o .

# compile each one, then verify the result
python3 annotate_tex.py build -o .

# all four, after clearing stale aux files
python3 annotate_tex.py all --clean-aux -o .
```

`build` runs `pdflatex`, `bibtex`, `pdflatex`, `pdflatex` and then checks the result rather than trusting the exit status: undefined refer-

ences, LaTeX errors, the note count against the ledger, and whether the PDF is actually newer than its source. It exists because one pdflatex pass leaves every cross-reference reading ?? and make calls that a success.

A defect in the package fails the build; a defect in the paper does not. A reference whose label exists in the source but hasn't resolved means the build has not converged, and fails. A reference whose label exists nowhere can never resolve, prints ?? in the authors' own PDF, and is reported as a note. The paper is allowed to be broken — that is what the package is for. Getting this backwards is how a make that has never once run to completion ships as the documented rebuild path.

Where a paper's numbering is self-referential, run build twice and read what it reports. A fixed number of passes is not a guarantee.

Writing notes

Note bodies are Markdown, because the issue list is Markdown, and the generator translates them for the LaTeX path. Quotation marks become directional quotes, emphasis becomes `\emph`, backtick spans become `\texttt`, and the characters that would otherwise break LaTeX are escaped. Math between dollar signs passes through untouched, so a quotation containing mathematics still pairs correctly.

Do not hand-write LaTeX quoting in a note — it reaches the Markdown literally.

Two forms for the entries themselves. Bulk copy-editing goes in a table: thirty one-line usage corrections as thirty paragraphs is far worse to read and act on than one three-column table of location, what the paper writes, and what to write instead. Prose entries are reserved

for items that need an argument. And when one item inside a table section carries real consequence — the garbled sentence that happens to be the one completing the proof — it gets lifted out of the table into a full entry, keeping its tag.

Adjacent one-line nits in the same paragraph can be bundled into a single annotation box rather than stacking three boxes each a line tall. They stay separate entries in the list.

What is written by hand

Three documents. The rest is generated.

The summary is the part you read first and lean on hardest, since it is where the paper is characterized rather than itemized. Four parts, in order: what the paper does, with the main theorem stated in full and the shape of the proof in as many steps as it actually has; the context, with the precise trade against the nearest prior work; the significance, with reasons, and set against it what qualifies the assessment — how much is assembly, what the hypotheses cost, what the result does not give; and the findings, ordered by consequence and cross-referenced to their tags, together with what checked out and what was never checked.

It ends there. No fifth section, no recommendation.

The repairs document covers issues with a clear route, where the route is short enough to state inline. It opens by saying what it does not claim — that every downstream calculation has been rechecked under each replacement hypothesis.

The README says what each file is, how to rebuild, and which of the three source cases you're in.

It is called a guide everywhere it is named — the title, the running head, the README heading, any covering note. *A critical guide* to the paper, never *a report on* it. The wording is what tells a reader at a glance that the document carries no verdict, and it is the first thing that drifts.

The finished package

```
critical-guide/
  00-guide.pdf          summary + issue
    list
  01-summary.md         hand-written prose
  02-issues.md          generated, point by
    point
  03-repairs.md         repairs with a
    clear route
  repairs/              the multi-route
    dossier
  issues.yaml           the ledger
  annotated-<cat>.tex    one per category
  annotated-<cat>.pdf    compiled, colour-
    coded
  annotate_tex.py       copied in
  Makefile
  README.md
  guide-preamble.tex
  guide-meta.yaml
```

review-notes/ sits beside it and is not part of the handover. It is the audit trail — the sweeps, the refutations, the findings that were dropped and why — kept so that a finding you doubt can be traced back to how it was reached.

The gates

The build is verified, never assumed. Five things are checked before the package is handed over:

1. `make` exits zero from a clean tree. Not `make guide`, not the tool run by hand. It is the documented rebuild path and the one thing you will actually run.
2. Every annotated document reports `ok` from the generator's own build check.
3. The guide PDF passes the same mechanical gate, not a lesser one. It is the document you actually read and the one that fails quietly — an undefined citation warns rather than errors, and unresolved citations can survive a visual spot-check.
4. The issue list's tables break across pages with their headers repeating. Nothing checks this automatically; a page is rendered and looked at.
5. In Case B, every annotated passage was confirmed present in the submitted PDF.

Then the hand-back: where the package is, what it contains, the count of findings by category and grade, what the refutation pass dropped, and what was not verified. Every quantity in that report was measured from the files — not carried over from an earlier run's figures.

Long Work, and Corrections

Two situations the ordinary flow doesn't cover: a work too long to examine in one pass, and a guide that turns out to need fixing after it's finished.

When one pass isn't enough

The question to ask before anything else — because it changes everything after — is whether a reader would take this work in one sitting and form one judgment.

A paper, however long, yes. A book, a thesis, a memoir in parts, no. Those get examined a chunk at a time, in separate sessions, with you reading each chunk's output before declaring the next. Forty pages is not chunked. Three hundred is. In between, the sitting-and-judgment test decides.

The pause between chunks is the point. It keeps you in control of a long examination rather than handing you one unreadable delivery at the end.

Chunking also settles a tooling question. The generator annotates one source file, so a `main.tex` full of `\include` lines would fail on every anchor, since the anchors live in the chapter files. Under chunking each chunk's source *is* its chapter file and the problem doesn't arise.

Reconnaissance first

Before any chapter is examined there's a chunk 0: front matter, table of contents, introduction, bibliography, and a skim for structure. No deep reading.

It produces the carry file with the work's structure and the decisions filled in, and nothing else. It's cheap, and it stops chunk 1 from doing double duty as both a chapter examination and the establishment of the frame that every later chunk inherits. Categories, colours, reader profile, and voice are fixed here and do not drift afterwards.

The carry file

`review-state.yaml` lives beside the work, and it is the whole design. Nothing survives between sessions except what is written into it — a fresh agent starting chunk 5 knows only what that file says.

Block	What it holds
work	title, authors, identifier, kind, source root
structure	one entry per chunk: file, pages, tag prefix, status, where its package landed
decisions	reader profile, voice, categories and colours — written once at chunk 0 and never edited
conventions	notation introduced where, and what it means
inherited	open defects from earlier chunks that later results rest on
precedents	grading calls that needed thought, and why they went the way they did
verified_clean	the citation cache: what has been checked and what it said
revisions	problems in an earlier chunk found while reading a later one
not_examined	cumulative, and honest

Four of those carry most of the weight.

inherited is what earns the design. Before anything is graded, the chunk checks whether the result it's examining rests on an open defect from an earlier chunk. If it does, the finding is *provisional* rather than independent, and says so — the same distinction the ledger's dependency field makes within a chunk, carried across chunks.

decisions is frozen. If a later chunk finds the category scheme inadequate it says so in the hand-back and lets you decide; changing it mid-examination makes the packages mutually inconsistent.

precedents is what keeps ten chapters examined by ten agents from disagreeing about what *major* means. Not every grade goes in it — only the ones where a reasonable reader could have gone the other way.

revisions is appended to, never acted on. A chunk-5 agent that discovers a chapter-1 problem records it and stops. Whether to re-run chapter 1 is your call, and re-running is expensive.

What changes per chunk

The pipeline runs as written, scoped to the chunk, with four differences.

The carry file is read in full, before anything else.

The context sweep does not run. Where the work sits, what's new, how it compares to prior work: none of that is knowable from chapter 2, and a chunk that attempts it produces confident nonsense. It belongs to the final synthesis.

The reference sweep consults the cache first. A book cites the same works many times; only what's new gets verified, and what's checked gets added.

Tags are namespaced by chunk — M1-7 is the seventh mathematical issue in chunk 1. No collisions, no high-water mark to track, and a tag says where it lives.

At the end the carry file is updated and the run stops. It does not begin the next chunk.

The synthesis

After the last chunk, one pass over the accumulated state produces what no chunk could: the summary with context and significance, the global assessment, the findings that cross chapters, and a consolidated issue list.

Correcting a finished guide

A finished guide is not frozen. A claim turns out to be overstated, someone disputes a finding, a cited preprint appears in print, the authors post a new version and the anchors stop matching.

When that happens, do not start a new run. The guide is the product, and the job is to correct it in place so that what you're holding is right. Three phases, and the order is what keeps a correction from becoming a rewrite.

Verify first, changing nothing

Enumerate the claims in scope, quote each verbatim with its location, and attack them one at a time — opening the cited work rather than relying on its abstract, on what the paper under examination says about it, or on the field's reputation of it. A verdict and its evidence are committed for every claim *before* any prose is touched.

An agent allowed to edit while it reads will drift into rewriting what it merely likes better. That is the failure this ordering exists to prevent, and it looks like productivity while it happens.

Negative and priority claims take a different standard. “No prior occurrence,” “the first in arbitrary dimension,” “only the surface case was known”: none of these can be re-verified by failing to find a counterexample again. What search would surface one is stated, run, and reported. A thin original basis means the claim gets **weakened**, not re-affirmed.

Then apply, narrowly

Each failed claim is corrected in the summary, in the guide’s existing voice — and corrected in the ledger too, wherever it also appears there. A literature claim frequently lives in both, and fixing only the prose leaves the guide contradicting its own annotated PDFs.

An honest hedge beats a deletion. A claim narrowed to what is actually true is more useful than a gap where it used to be. Nothing that wasn’t audited gets touched, and no recommendation gets added.

Then regenerate, and report the changelog

```
python3 annotate_tex.py all --clean-aux
```

Then the guide PDF. A correction that leaves the ledger and the built PDFs out of step has not been made — it has been half made, which is worse, because the package now disagrees with itself.

Finally: what changed, from what to what, on what evidence. The detail stays in the audit trail.

The same three phases apply to any correction, not only to a literature

claim — a withdrawn finding, a regraded issue, a repaired anchor after a new version appears.

Quick Reference

Invocation

```
/write-critical-guide <pdf path>  
/write-critical-guide arXiv:<id>  
/write-critical-guide <folder with the  
paper>
```

It runs unattended and takes every default. The only thing that halts it is being unable to identify the paper.

The generator

```
# anchors only; writes nothing  
python3 annotate_tex.py check -v  
  
# annotated .tex, one per category  
python3 annotate_tex.py annotate -o .  
  
# the Markdown issue list  
python3 annotate_tex.py issues-md -o .  
  
# compile, then verify the result  
python3 annotate_tex.py build -o .  
  
# all four, aux files cleared first
```

```
python3 annotate_tex.py all --clean-aux -o
.

# the same gate, for 00-guide.pdf
python3 check_guide.py
```

Option	Effect
-l PATH	ledger path; defaults to issues.yaml
-o DIR	output directory
--clean-aux	clear stale aux files first; use it after a failed build
-v	verbose; use it with check

Package layout

File	What it is
00-guide.pdf	summary and issue list, typeset together
01-summary.md	hand-written: work, context, significance, findings
02-issues.md	generated: point by point
03-repairs.md	repairs with a clear route
repairs/	conditional: the multi-route dossier
issues.yaml	the ledger; single source of truth
annotated-*.tex	generated, one per category
annotated-*.pdf	compiled, colour-coded
annotate_tex.py	copied in; the package rebuilds anywhere
Makefile	make is the documented rebuild path
review-notes/	the audit trail; not part of the handover

Ledger fields

Required per issue: `id`, `category`, `anchor`, `note`. Everything else is optional, and `grade` defaults to `minor`.

Field	Meaning
<code>id</code>	the tag: <code>M-7</code> , <code>R-3</code> , <code>T-21</code>
<code>category</code>	which category; they partition
<code>grade</code>	<code>major</code> , <code>minor</code> , <code>trivial</code>
<code>grade_note</code>	qualifier, e.g. <i>major (for readability)</i>
<code>location</code>	where, in the paper's own terms
<code>anchor</code>	verbatim substring of exactly one line of the source
<code>occurrence</code>	which match, when the anchor is deliberately non-unique
<code>caption</code>	short label for the note and the header index
<code>quote</code>	the paper's own words; issue list only
<code>note</code>	the finding itself
<code>request</code>	what would repair it
<code>depends_on</code>	this item follows from another finding
<code>confidence</code>	<code>direct-check</code> , <code>assessment</code> , <code>reported</code>
<code>also</code>	bundle these tags into this one's annotation box
<code>render</code>	<code>prose</code> lifts an item out of a table section

Top-level blocks of the ledger:

paper	the work; what locations
mean	
categories	tag, title, colour,
output	
issues	the findings themselves
verified_clean	what came out clean
not_verified	what was not checked, and
why	
scope_caveat	what is not claimed
certified	
issue_list_name	the issue list's filename
caption_symbols	ASCII spellings for
captions	

Where fields appear

Field	Issue list	Inline note	Header index
id	yes	yes	yes
grade	yes	yes	yes
location	yes	—	fallback
caption	—	caption	yes
quote	yes	—	—
note	yes	yes	—
request	yes	yes	—
depends_on	yes	—	—
confidence	yes	—	—
anchor	—	placement	—

The annotated source omits the quotation on purpose: the note sits at the passage, so quoting it back is noise. The issue list includes it, since it's read away from the paper.

Source cases

Case	What is annotated, and what locations mean
A	The authors' source. Locations cite it by line, plus the paper's numbering.
B	A public source, with every passage confirmed against the submitted PDF. Prose locations are the submission's pages and lines.
C	Nothing; the package is prose and the typeset guide. Stated in the README.

LaTeX hazards, already handled

Recorded so that placing a note by hand, or debugging a build, doesn't mean rediscovering them.

Hazard	What happens
A list of todos under amsart with hyperref	aborts the build, naming neither the package nor the caption; an inline index replaces it
TeX inside a note caption	breaks hyperref; captions are reduced to plain ASCII
A note dropped after its anchor line	can land inside math; insertion advances past all math and verbatim
A stale todo file	resurfaces as a phantom error after a failed run; clear aux files
A percent or hash in a quoted macro	comments out the rest of the note, including its closing brace
A quoted macro in a narrow cell	one unbreakable box; overfull is a warning, so the log won't stop you
A note beginning with a bracket	turns a line break into a length argument, and the length is not a length
pdflatex log encoding	is not promised to be UTF-8; read it permissively
Missing figures or class files	not yours to fix; note it and let the build proceed

Building the guide PDF

Bare LaTeX macros quoted from the paper must sit in backticks in the Markdown, or they reach an engine that has never heard of them. This is the commonest way the build breaks, because quoting the paper's macros is exactly what an issue list does.

Use pdflatex, not lualatex — an order of magnitude faster here, and it handles every character a critical guide actually uses. Long tables need longtable with repeating headers and ragged-right cells, or they are silently truncated. And the title lives in the metadata file; repeating it as a heading prints it twice.

Chunked examinations

Element	Rule
<code>review-state.yaml</code>	beside the work; read in full before starting, updated before stopping
Chunk o	reconnaissance only; fixes categories, colours, profile, voice
decisions	written once, never edited
Context sweep	does not run per chunk
Reference sweep	consults the cache first
Tags	namespaced by chunk: M1-7
revisions	appended, never acted on
End of chunk	update, hand back, stop

Words the house does not use

Disposition vocabulary as a judgment on this paper: *accept, reject, revision, publishable*. Anything that writes your report for you: *I recommend, this reviewer finds*. *Load-bearing* — say what rests on what. *The key definition, the crucial step* — there is rarely exactly one; name the thing and say what it does. Negative framing where positive carries the same content. Executive-summary register: *headline, takeaway, at a glance*. Reading advice to the author. And the usual filler: *utilize, leverage, seamless, robust, powerful, elegant, simple, easy, straightforward, clean*.